

Unified Ranking Across Retrieval Channels: A Large-Scale E-Commerce Approach

Aditya Gaydhani*
Data Sciences, Target Corporation
Brooklyn Park, MN, USA
aditya.gaydhani@target.com

Guangyue Xu*
Data Sciences, Target Corporation
Brooklyn Park, MN, USA
guangyue.xu@target.com

Dhanush Kamath
Data Sciences, Target Corporation
Sunnyvale, CA, USA
dhanush.kamath@target.com

Ankit Singh
Data Sciences, Target Corporation
Sunnyvale, CA, USA
ankit.singh2@target.com

Alex Fengjie Li
Data Sciences, Target Corporation
Sunnyvale, CA, USA
alex.li@target.com

Abstract

Large-scale e-commerce search and recommendation systems rely on multiple specialized retrieval channels to surface diverse products, each optimized for a distinct objective. A key challenge is merging candidates from these heterogeneous channels into a single ranked list under strict latency constraints while optimizing business KPIs such as user conversion. Static rank-fusion methods such as Reciprocal Rank Fusion (RRF) and Weighted Interleaving use the same fusion behavior across queries, limiting their ability to adapt to context or model interactions among channel signals. We reformulate multi-channel fusion as a channel-aware learning-to-rank problem and propose a unified ranking model trained on a scalar, funnel-weighted graded-relevance label that combines clicks, add-to-carts, and purchases. The model incorporates channel-specific scores together with recent aggregate query-item behavior to capture near-term population demand. The evaluated channel set combines conventional relevance retrieval with query-conditioned, recommendation-oriented channels emphasizing newness, trending demand, regional relevance, and seasonality. A randomized online experiment shows a 2.85% increase in user conversion over the deployed weighted-interleaving control. The model achieves a p95 reranking-stage latency below 50 ms, and has been deployed on Target.com.

CCS Concepts

• **Information systems** → **Combination, fusion and federated search**; *Rank aggregation*; *Learning to rank*.

Keywords

Rank Fusion, Learning-to-Rank, Multi-Channel Retrieval

*These authors contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '26, Minneapolis, MN

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-X-XXXX-XXXX-X/YYYY/MM
<https://doi.org/XXXXXXXX.XXXXXX>

ACM Reference Format:

Aditya Gaydhani, Guangyue Xu, Dhanush Kamath, Ankit Singh, and Alex Fengjie Li. 2026. Unified Ranking Across Retrieval Channels: A Large-Scale E-Commerce Approach. In *Proceedings of 20th ACM Conference on Recommender Systems (RecSys '26)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/XXXXXXXX.XXXXXX>

1 Introduction

Large-scale e-commerce systems consist of catalogs containing millions of products spanning diverse categories, price ranges, and life-cycle stages. Production search and recommendation systems operate as a multi-stage pipeline comprising retrieval, ranking, and re-ranking. Because it is computationally infeasible to score the entire catalog for every request under strict latency budgets, retrieval first selects a manageable candidate set of relevant products, after which downstream ranking stages refine the top- k results to optimize user-facing metrics. To balance user satisfaction and business objectives, modern systems rely on multiple specialized retrieval channels, each optimized for a distinct objective (e.g., lexical relevance, semantic similarity, freshness, or seasonality) [8, 19]. In addition to lexical and semantic retrieval, channels emphasizing newness, trending demand, regional relevance, or seasonality act as query-conditioned recommendation-style candidate generators. They remain conditioned on the search query while emphasizing complementary discovery objectives. The final ranked list is produced by merging candidates from all of these heterogeneous channels.

Although multi-channel retrieval improves recall and diversity, it introduces significant re-ranking challenges. Channels are optimized independently and generate candidates with heterogeneous score distributions, bias characteristics, and objective functions. For instance, popularity-driven channels emphasize long-term engagement, whereas freshness-driven channels favor recency and exploration. Channel utility varies with query context and time, and the final ranking must optimize downstream business KPIs such as user conversion. These modeling challenges are further constrained by strict latency requirements, as even modest increases in response time can negatively impact user satisfaction and engagement [1]. As a result, designing a fusion strategy that models context-dependent channel utility and cross-channel interactions under tight computational budgets remains technically challenging for e-commerce.

Prior static fusion methods such as RRF [3] and Weighted Interleaving [7, 22] apply query-invariant fusion rules, limiting their ability to adapt channel influence to the query or exploit joint evidence across sources. Existing LTR approaches [10, 11] are typically designed for single-channel candidate distributions and do not explicitly model heterogeneous retrieval sources. We address this gap with a unified, channel-aware LTR framework (Section 3.1). We formulate multi-channel fusion for e-commerce search as a context-dependent learning-to-rank problem over heterogeneous query-conditioned candidate sources. Instead of relying on a single global fusion rule, the downstream ranker uses channel scores together with item and aggregate query-item features. The upstream channels retain their distinct objectives, while the final ranker optimizes one scalar business-aligned label that combines clicks, add-to-carts, and purchases. Recent aggregate behavioral signals reflect changing population-level demand rather than individual session intent. The resulting model places relevance-oriented and discovery-oriented channels under a common production reranker while meeting strict latency constraints in high-traffic production environments.

Our main contributions are summarized as follows:

- We formulate and deploy a practical cross-channel ranking framework that replaces globally tuned fusion over independently ranked, query-conditioned channel outputs with a single learning-to-rank model.
- We propose a channel-aware representation over a fixed production channel set using each upstream channel’s score and membership features.
- We introduce a weekly funnel-weighted graded-relevance label that scalarizes clicks, add-to-carts, and purchases into one conversion-aligned ranking objective.
- Through a large-scale online A/B experiment, we show a +2.85% improvement in user conversion over the deployed weighted-interleaving control while meeting a p95 reranking-stage latency below 50 ms.

2 Related Work

Rank Fusion and Multi-Channel Retrieval. Reciprocal Rank Fusion (RRF) [3] and Weighted Interleaving [7, 22] are widely adopted strategies for merging candidates from multiple retrieval channels. Standard RRF combines source ranks through a fixed reciprocal-rank transformation, whereas Weighted Interleaving samples from independently ranked lists using globally tuned channel-selection weights. Neither method conditions its fusion behavior on the query or models interactions among channel signals. Huang et al. [8] systematically study multi-channel fusion at the retrieval stage and show that optimized channel weighting improves retrieval quality; however, their work focuses on retrieval-level optimization and does not address how heterogeneous candidates should be jointly modeled by a downstream ranking model. Our work addresses this gap by learning a unified scoring function directly over the merged candidate pool.

Learning to Rank in Industrial E-Commerce. LTR models are central to industrial search systems [10, 11]. Despite rapid advances in deep learning, tree-based models such as Gradient Boosted Decision Trees (GBDT) remain widely deployed due to their efficiency

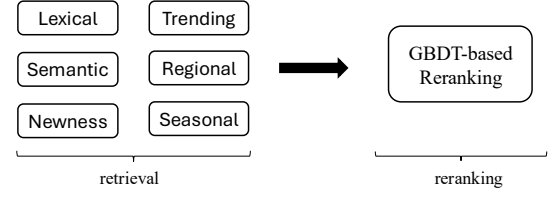


Figure 1: Overview of the unified ranking system. Multiple channel-specific retrieval components independently produce ranked candidate sets, which are merged and re-ranked by a single GBDT-based LTR model using channel-aware and engagement features.

and robustness under realistic latency constraints [9, 17]. Prior work [21] has demonstrated the value of incorporating clicks as both training labels and input features; we extend this idea to aggregate click, add-to-cart, and purchase signals in a multi-channel setting. Existing LTR work typically assumes a single homogeneous candidate distribution and does not explicitly model heterogeneous retrieval sources. In our setting, the upstream channels are optimized for distinct objectives, while the downstream LambdaMART model is trained on one scalar business-aligned objective.

System-Level Optimization across Ranking Stages. Several works explore coordinated optimization across pipeline stages. Qin et al. [16] propose a multi-agent framework for e-commerce search that models interactions among retrieval and ranking components. While these approaches emphasize cross-stage coordination, they do not study unified ranking over heterogeneous multi-channel candidates under strict production latency constraints.

Unified Modeling across Search and Recommendation. A related line of research investigates joint modeling of search and recommendation. Liu et al. [13] propose a multi-scenario learning framework that shares representations across search and recommendation by exploiting a global label space. Such methods focus on representation learning and multi-task optimization, assuming a fixed candidate generation process. Our work instead studies production search, where newness, trending, regional, and seasonal components provide query-conditioned recommendation-style candidates alongside lexical and semantic retrieval.

3 Methodology

3.1 Problem Formulation

A large-scale e-commerce search system serves millions of queries daily across a catalog of tens of millions of products. Given a query $q \in \mathcal{Q}$, the system retrieves and ranks items from a large corpus $\mathcal{I}$. A single retrieval channel is insufficient to satisfy the diverse informational needs of users: lexical matching excels at exact queries, while semantic retrieval handles paraphrases and long-tail intent. The system therefore employs a set of K heterogeneous retrieval channels

$$\mathcal{C} = \{c_1, c_2, \dots, c_K\},$$

where each channel c_k is optimized for a distinct objective (e.g., lexical relevance, semantic similarity, item freshness, or long-term popularity).

For a given query q , each channel c_k first applies its own production retrieval and ranking logic and independently produces a ranked list

$$\mathcal{R}_k(q) = \{(i, s_k(q, i)) \mid i \in \mathcal{I}\},$$

where $s_k(q, i)$ denotes the channel-specific retrieval score. Because scoring all retrieved items in downstream stages is computationally infeasible, only the top- n_k items from each channel are forwarded to re-ranking. We denote this truncated list as

$$\mathcal{R}_k^{(n_k)}(q) = \text{Top}_{n_k}(\mathcal{R}_k(q)).$$

The final candidate pool for re-ranking is defined as the union of these truncated lists:

$$\mathcal{R}(q) = \bigcup_{k=1}^K \mathcal{R}_k^{(n_k)}(q).$$

Note that items may appear in multiple channels; duplicates are deduplicated in $\mathcal{R}(q)$, with all channel-specific scores $\{s_k(q, i)\}$ retained as features for the re-ranking stage rather than being aggregated at this step.

The re-ranking stage learns a scoring function

$$f(q, i; \theta) : \mathcal{Q} \times \mathcal{I} \rightarrow \mathbb{R},$$

which assigns a unified relevance score to each (q, i) with $i \in \mathcal{R}(q)$. Items are sorted in descending order of $f(q, i; \theta)$ to produce the final ranked list. The current model is trained and deployed for query-driven search and assumes a fixed production channel set $\mathcal{C}$ available during training. The scoring function f is learned from historical interaction data; the following sections describe the training data representation, label construction, and features used in production.

3.2 Data Representation

All training instances are defined at the *query–item–week* level to incorporate temporal dynamics into the ranking process. Let t_w denote the beginning of week w . The label for (q, i, w) is computed from interactions occurring during $[t_w, t_{w+1})$, while every behavioral feature for that instance is computed exclusively from events before t_w . For example, the 7-day feature window is $[t_w - 7 \text{ days}, t_w)$, and the 14-day and 30-day windows end at the same cutoff. Feature and label windows therefore do not overlap.

Weekly aggregation was chosen empirically as a balance between responsiveness and statistical stability. Daily windows are too sparse and noisy for tail queries, while bi-weekly windows respond more slowly to seasonal, promotional, and emerging demand. Interaction counts and labels are reset at the beginning of each week rather than accumulated over the lifetime of a query–item pair. This limits the carryover of stale historical popularity and allows current population demand for new, trending, and seasonal products to affect the training target.

Modeling temporal structure is important in multi-channel retrieval settings, where channels may emphasize distinct objectives such as long-term popularity, short-term trendiness, or freshness. To capture both stable and emerging patterns, we incorporate behavioral features computed over multiple lookback windows ending at t_w [14], including long-term aggregates (e.g., historical sales and engagement) and short-term signals that capture population-level demand velocity. This design limits reliance on lifetime popularity

while allowing query-dependent, time-sensitive channel utility. It does not enforce channel-level exposure quotas. Given this temporal data structure, we next define the supervision signal used to train the ranking model.

3.3 Label Construction

User interactions in e-commerce search follow a conversion hierarchy:

$$\text{Impression} \rightarrow \text{Click} \rightarrow \text{AddToCart} \rightarrow \text{Purchase}.$$

For each query–item pair within a weekly window (q, i, w) , we summarize engagement over N sessions using the deepest action per session. Let $\{V_{q,i,w}, C_{q,i,w}, A_{q,i,w}, P_{q,i,w}\}$ denote counts of *view-only*, *click*, *add-to-cart*, and *purchase* outcomes, respectively, with $N = V_{q,i,w} + C_{q,i,w} + A_{q,i,w} + P_{q,i,w}$. We define a scalar funnel-weighted graded-relevance label as a weighted aggregation:

$$L(q, i, w) = aP_{q,i,w} + bA_{q,i,w} + cC_{q,i,w} + dV_{q,i,w},$$

where $a \geq b \geq c \geq d \geq 0$. We calibrate weights using corpus-level conversion statistics aggregated over the training period:

$$a = 1, \quad b = \frac{|\mathcal{P}|}{|\mathcal{A}|}, \quad c = \frac{|\mathcal{P}|}{|\mathcal{C}|}, \quad d = 0,$$

where $|\mathcal{P}|$, $|\mathcal{A}|$, and $|\mathcal{C}|$ denote total purchases, add-to-carts, and clicks in the training corpus. Setting $d = 0$ excludes view-only impressions from the label, as they do not indicate purchase intent and would dilute the conversion signal—the primary business objective for Target’s search system.

To ensure comparability across queries, we apply per-query max normalization:

$$\tilde{L}(q, i, w) = \frac{4L(q, i, w)}{\max_{i' \in \mathcal{R}(q, w)} L(q, i', w)},$$

mapping labels to $[0, 4]$. The factor of 4 aligns with the standard graded relevance scale used in LambdaMART training, where higher values correspond to more relevant items (e.g., $\tilde{L} = 4$ for the most-purchased item per query). We normalize at the query level rather than globally across all queries, as global normalization would conflate queries with vastly different engagement volumes and introduce high variance for sparse queries—a common challenge in long-tail e-commerce search traffic [20]. Engagement counts and labels are reset for each weekly window rather than accumulated over the lifetime of the query–item pair. The target is therefore designed to optimize realized weekly business value rather than estimate a stationary per-impression conversion propensity. This limits long-horizon popularity carryover and allows newly released, trending, regional, or seasonal candidates to gain relevance when they attract current-week engagement and conversion, although it does not impose channel-specific exposure guarantees. With labels defined, we describe the features used to represent each query–item–week instance.

3.4 Features

We construct ranking features at a weekly granularity (Section 3.2) and group them into three categories.

Item Features. These capture intrinsic product attributes (e.g., price, category, brand, availability) and behavioral aggregates over multiple lookback windows (e.g., 7-day, 14-day, 30-day), all ending before the beginning of the label week. The features model long-term popularity, recency, and seasonality, enabling the model to balance historically strong performers with newly trending or seasonal products.

Channel-Aware Query–Item Features. For each query–item pair, channel-aware features include the raw retrieval score $s_k(q, i)$ from each channel. Because channel scores reflect distinct upstream objectives, these features provide the model with signals from which to learn query-dependent channel utility and cross-feature interactions. Items absent from a given channel receive NA values for the corresponding features, which are handled natively by the GBDT model without imputation.

Query–Item Engagement Features. Extending prior work on using clicks as both labels and features [21], we incorporate clicks, add-to-cart events, and purchases as engagement features. These features are computed strictly before t_w and use a weighted formulation similar to Section 3.3, with distinct aggregation and temporal decay to capture short-term population demand velocity. For feature computation, data is aggregated across users and across all query variations identified by our query understanding models (e.g., spelling corrections and query rewrites), ensuring robust signal aggregation for semantically equivalent queries and improving coverage for head and torso queries. These signals capture aggregate query–item demand. Their impact is evaluated in Section 5.2. These features are used to train a GBDT-based ranking model, described next.

3.5 Model and Training

We adopt Gradient Boosted Decision Trees (GBDT) as the ranking model for three reasons specific to this production setting. First, GBDT handles heterogeneous feature types—continuous retrieval scores, categorical channel indicators, and sparse engagement signals—without requiring feature normalization. Second, it supports native NA handling for missing channel features, eliminating the need for imputation. Third, it provides deterministic and efficient inference, critical for meeting the p95 <50 ms reranking-stage latency requirement under high-traffic production conditions [4, 9]. In large-scale production search systems, tree-based models remain competitive with deep neural approaches under realistic latency and feature constraints, particularly when feature engineering plays a central role [9, 17]. GBDT constructs an additive ensemble of decision trees:

$$f(q, i; \theta) = \sum_{t=1}^T \eta \cdot h_t(q, i),$$

where h_t is the t -th tree, T is the total number of trees, and η is the learning rate (shrinkage factor). Each tree is trained sequentially to minimize the residual loss from previous iterations. Our model is implemented using the Yggdrasil Decision Forests library [6], which provides efficient training and inference optimized for high-throughput serving environments.

We train the model using the LambdaMART objective [2], whose pairwise gradient updates are weighted by changes in NDCG. This

aligns naturally with our graded relevance labels $\tilde{L} \in [0, 4]$ derived from the conversion hierarchy and enables the model to prioritize high-value purchase events while still accounting for clicks and add-to-cart signals.

To effectively model heterogeneous retrieval signals and behavioral features, we employ a local tree growing strategy with sparse oblique splits (linear combinations of features at each split node), enabling efficient handling of correlated and high-dimensional feature spaces such as multi-channel retrieval scores. Regularization is applied through L2 penalties and shrinkage to control model complexity and mitigate overfitting. We enable second-order (Hessian-based) gain computation to improve split quality and stabilize training. Hyperparameters—including tree depth, number of trees, learning rate (shrinkage), and minimum leaf size—are tuned via grid search on the held-out fourth-week validation set described in Section 4.1. Final model selection is based on offline NDCG@8 on the validation set.

4 Experiment Design

All experiments in this paper are conducted in an e-commerce search system. The evaluated candidate pool combines lexical and semantic retrieval with channels that emphasize newness, trending demand, regional relevance, and seasonality while remaining conditioned on the search query. These channels play a recommendation-like discovery role within the search pipeline. The results therefore concern a common reranker over heterogeneous query-conditioned channels, not a model shared between search and recommendation surfaces.

4.1 Dataset

Our experiments are conducted using large-scale user interaction logs collected from the Target’s e-commerce search system, in accordance with internal privacy, security, and data governance policies. The dataset includes search queries, ranked item impressions, and corresponding user interactions, including clicks, add-to-cart events, and purchases, as defined in Section 3.3. We use a recent five-week period to construct labeled examples. The dataset is partitioned chronologically: the first three label weeks are used for training, the fourth label week for validation and hyperparameter tuning, and the final label week is held out for offline evaluation. Behavioral feature windows may draw on historical events preceding this five-week label period, but always end strictly before the corresponding label week. This temporal split enables evaluation under concept drift and non-stationarity without overlapping feature and label windows.

To improve data quality and reduce noise from extreme long-tail queries, we filter the data at the query-week level, retaining only queries for which at least one item receives at least one purchase within the same week. For each retained query, we include up to the top- n_k items per channel for reranking, as described in Section 3.1, to prevent train–inference skew. The final training set comprises approximately 60M rows and 500k unique queries distributed across head, torso, and tail segments. Queries are segmented using production query-frequency thresholds into head, torso, and tail groups, each accounting for 33% of search traffic.

Labels and features are computed for each query–item–week instance as described in Sections 3.3 and 3.4, respectively.

4.2 Model Variants

We evaluate four model variants to isolate the contribution of each component. UR and UR + EF use the same production graded-relevance label, whose coefficients are determined heuristically based on internal business requirements. Holding that label fixed allows the comparison to isolate the addition of engagement features.

- **WI (Baseline).** Each channel first applies its own production retrieval and ranking logic using channel-specific features and objectives. The deployed Weighted Interleaving layer then constructs the final ranking by probabilistically sampling from these independently ranked lists according to globally tuned channel-selection weights [7, 22]. We use this system as the control because it is the production fusion mechanism being replaced; an exhaustive benchmark of static rank-fusion algorithms is outside the scope of this work.
- **UR.** Unified ranking using the same upstream channel outputs and a GBDT model trained on item features and channel-aware query–item features (Section 3.4). This variant replaces only the final weighted-interleaving fusion layer with a learned cross-channel reranker.
- **UR + EF.** Unified ranking with engagement features. This variant extends UR by incorporating aggregate query–item clicks, add-to-cart events, and purchases over multiple pre-week time windows (Section 3.4). It uses the same production label as UR, so the comparison isolates the contribution of these behavioral features.
- **UR + EF + CL.** Unified ranking with engagement features and the conversion-weighted label. This is the full proposed model. It retains the architecture and features of UR + EF but replaces the production label with the scalar funnel-weighted label described in Section 3.3.

4.3 A/B Test Setup

We evaluated all ranking variants through a multivariate online A/B experiment on the production search platform. Traffic was randomly assigned at the visitor level, with each variant receiving 5% of eligible search traffic. The experiment ran for two full weeks and included all users, with bot traffic filtered out. The evaluated variants were WI (control), UR, UR + EF, and UR + EF + CL. We report relative lifts over the control for click-through rate (CTR), add-to-cart rate (ATC), and conversion rate. Statistical significance was assessed using a two-sided Student’s *t*-test at the 95% confidence level. All significance markers compare an individual treatment arm with the WI control. Prior to the experiment, a power analysis assuming 80% power, a 95% confidence level, and a 1% minimum detectable effect in conversion estimated the required sample size for significance.

5 Results

Table 1 summarizes the offline and online performance of all model variants. Offline NDCG@8 is used as a development and model-selection metric against held-out behavior-derived labels as the top-8 slots receive majority of the user engagement. Conclusions

Table 1: Offline and online performance comparison of model variants. Online metrics are reported as lift (%) over the weighted-interleaving control. * indicates statistical significance at the 95% confidence level for the corresponding treatment-versus-WI comparison.

Model Variant	Offline	Online (A/B Test Lift %)		
	NDCG@8	CTR	ATC	Conversion
WI (Baseline)	0.6620	–	–	–
UR	0.7169	+0.26	+1.21*	+1.28*
UR + EF	0.7799	+1.52*	+2.72*	+2.38*
UR + EF + CL	0.7994	+1.46*	+2.81*	+2.85*

about production impact are based on the randomized online A/B experiment. We discuss each ablation in turn. Additional qualitative examples illustrating changes in search ranking are provided in Appendix A.

5.1 Weighted Interleaving vs. Unified Ranking

We evaluate the unified ranking model against the weighted interleaving baseline shown in Table 1. In the baseline, independently ranked channel outputs are probabilistically merged according to globally tuned channel weights [7, 22]. Unified ranking receives the same upstream channel outputs but replaces only the final fusion layer with a query–item scoring function (Section 3.1) aligned with the downstream objective.

As reported in Table 1, unified ranking increases offline NDCG@8 from 0.6620 to 0.7169, indicating closer agreement with the behavior-derived development objective. Relative to WI, the online A/B test shows positive point estimates across all business metrics and statistically significant improvements in add-to-cart (+1.21%) and conversion (+1.28%). These findings show that replacing the deployed globally tuned fusion layer with a learned cross-channel reranker yields measurable production impact.

Figure 2 highlights the qualitative difference between weighted interleaving on the left and unified ranking on the right. The unified model places recent high-engagement products nearer the top of the ranking.

5.2 Contribution of User Engagement Features

We evaluate the contribution of query–item engagement features (Section 3.4) through an ablation study comparing the unified ranking model with and without these features.

As shown in Table 1, UR + EF increases offline NDCG@8 from 0.7169 to 0.7799. Relative to WI, it produces statistically significant lifts in CTR (+1.52%), add-to-cart (+2.72%), and conversion (+2.38%). These online point estimates are larger than those observed for UR and support the value of aggregate engagement features. The result indicates that historical query–item interactions provide a strong predictive signal for current population-level demand.

Figure 3 highlights the contribution of aggregate engagement features. Popular products from the semantic channel appear nearer the top of the grid, whereas the model on the left lacks these signals and relies more heavily on the upstream ordering. The unified

Table 2: Segment-level A/B test performance. Metrics are reported as lift (%) over the weighted-interleaving control. * indicates statistical significance at the 95% confidence level for the corresponding treatment-versus-WI comparison.

Model	Query Segment	CTR	ATC	Conversion
UR	Head	+0.76	+1.95*	+2.23*
	Torso	-1.09	+1.35*	+1.22*
	Tail	-1.13	+1.11*	+1.91*
UR + EF	Head	+2.70*	+4.28*	+4.39*
	Torso	+0.52	+2.27*	+2.87*
	Tail	-0.21	+1.93*	+1.89*
UR + EF + CL	Head	+2.92*	+4.76*	+5.27*
	Torso	-2.25*	+1.06*	+2.56*
	Tail	-2.23*	+0.38	+1.88*

model also surfaces a seasonal Halloween-themed product during October that is absent from the comparison ranking’s top 12.

5.3 Effectiveness of Conversion-Weighted Engagement Label

To isolate the impact of label construction, we compare a unified ranking model trained with the heuristic production label against an otherwise identical model trained with the proposed scalar conversion-weighted label (Section 3.3).

Table 1 shows that UR + EF + CL attains an offline NDCG@8 of 0.7994, compared with 0.7799 for UR + EF. Relative to WI, the full model produces statistically significant lifts in add-to-cart (+2.81%) and conversion (+2.85%) while maintaining a similar CTR lift. It has the highest observed conversion point estimate among the evaluated treatments. The findings indicate that scalar weighting of engagement outcomes across the conversion funnel aligns model training with downstream business value.

5.4 Query Segment Analysis

Table 2 reports online performance across head, torso, and tail query segments. All three unified-ranking variants produce statistically significant conversion lifts in every segment. Add-to-cart lifts are also statistically significant in every case except UR + EF + CL on tail queries (+0.38%). For head queries, UR + EF and UR + EF + CL significantly improve CTR, add-to-cart, and conversion, and the full model has the largest observed conversion lift (+5.27%). For torso and tail queries, UR + EF + CL produces significant CTR declines (-2.25% and -2.23%, respectively) while retaining significant conversion lifts (+2.56% and +1.88%). This pattern is consistent with the scalar conversion-weighted objective prioritizing downstream funnel outcomes even when CTR decreases.

6 Deployment and Serving

The unified ranking model is deployed within the production search stack. We train Gradient Boosted Decision Tree (GBDT) ranking models [4] using Yggdrasil Decision Forests (YDF) [6]. Offline feature pipelines built on Spark [23] generate the training data, and

a batch training job trains the YDF model before publishing the resulting artifact to Google Cloud Storage (GCS) [5].

For online serving, the production model is loaded and served through TensorFlow Serving [15]. Features are organized by update frequency into real-time, daily, and weekly feature sets. These features are materialized in Redis [18] to support low-latency lookup at request time. Real-time updates are consumed from Kafka topics [12] and continuously written to Redis, while daily and weekly features are refreshed by scheduled batch jobs that update the same serving store.

At request time, the Search Orchestration layer retrieves candidates from the available retrieval channels and passes them to the Ranking Orchestration layer. The Ranking Orchestration layer retrieves the required features from Redis, performs feature processing, calls the model-serving endpoint to score each candidate, applies business rules, and returns the final ranked response. In production traffic, the model achieves a p95 end-to-end latency of less than 50 ms, measured from feature retrieval through scoring computation across all candidates in $\mathcal{R}(q)$.

7 Conclusion and Future Work

We introduced a channel-aware learning-to-rank framework that replaces globally tuned fusion over a fixed set of independently ranked retrieval channels with a single downstream reranker. The model combines channel scores with aggregate pre-week behavioral features and optimizes a scalar weekly funnel-weighted label. Relative to the deployed weighted-interleaving control, the randomized online experiment produced statistically significant lifts in add-to-cart and conversion for the full model, including a 2.85% conversion lift, while maintaining a p95 reranking-stage latency below 50 ms.

We view the candidate-union and reranking abstraction as a search-side building block toward future unified search and recommendation. Future work includes adapting the framework to recommendation surfaces with surface-specific retraining, systematically evaluating bias and exposure across channels, and incorporating individual personalization signals beyond the population-level demand features studied here.

Acknowledgments

We thank Search — Data Science leadership at Target for their support in enabling this initiative. We are grateful to the Search Product team for their collaboration in formulating the case for incorporating user engagement features into the ranking framework, contributing to the A/B test design, and conducting the post-experiment analysis. We also thank the Query Understanding team for their work on query normalization, which improved the quality of the engagement features used in this study. Finally, we acknowledge the foundational contributions of the Search Retrieval and Engineering groups in developing the retrieval channels and ranker service that made this work possible, as well as the support of the Content Understanding and Search Platform teams.

References

- [1] Jake Brutlag. 2009. *Speed Matters for Google Web Search*. Technical Report. Google Inc. Google Technical Report.

- [2] Christopher J. C. Burges. 2010. *From RankNet to LambdaRank to LambdaMART: An Overview*. Technical Report. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/>
- [3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Boston, MA, USA) (SIGIR '09). Association for Computing Machinery, New York, NY, USA, 758–759. doi:10.1145/1571941.1572114
- [4] Jerome H. Friedman. 2001. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics* 29, 5 (2001), 1189–1232. doi:10.1214/aos/1013203451
- [5] Google. [n. d.]. Google Cloud Platform. <https://cloud.google.com/> Accessed: 2026-05-12.
- [6] Mathieu Guillaume-Bert, Sebastian Bruch, Richard Stotz, and Jan Pfeifer. 2023. Yggdrasil Decision Forests: A Fast and Extensible Decision Forests Library. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*. 4068–4077. doi:10.1145/3580305.3599933
- [7] Katja Hofmann, Shimon Whiteson, and Maarten de Rijke. 2011. A probabilistic method for inferring preferences from clicks. In *Proceedings of the 20th ACM International Conference on Information and Knowledge Management (CIKM '11)*. ACM, 249–258. doi:10.1145/2063576.2063618
- [8] Junjie Huang, Jiarui Qin, Jianghao Lin, Ziming Feng, Yong Yu, and Weinan Zhang. 2025. Unleashing the Potential of Multi-Channel Fusion in Retrieval for Personalized Recommendations. *arXiv preprint* (2025).
- [9] Junjie Huang, Jiarui Qin, Weinan Zhang, and Yong Yu. 2025. Industry Insights from Comparing Deep Learning and GBDT Models for E-Commerce Learning-to-Rank. In *Proceedings of the 19th ACM Conference on Recommender Systems (RecSys)*. Industry Track.
- [10] Md. Ahsanul Kabir, Mohammad Al Hasan, Aritra Mandal, Daniel Tunkelang, and Zhe Wu. 2024. A Survey on E-Commerce Learning to Rank. *arXiv preprint arXiv:2412.03581* (2024). doi:10.48550/arXiv.2412.03581
- [11] Shubhra Kanti Karmaker Santu, Parikshit Sondhi, and ChengXiang Zhai. 2017. On Application of Learning to Rank for E-Commerce Search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Shinjuku, Tokyo, Japan) (SIGIR '17). Association for Computing Machinery, New York, NY, USA, 475–484. doi:10.1145/3077136.3080838
- [12] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB '11)*. Athens, Greece.
- [13] Jinhan Liu, Qiyu Chen, Junjie Xu, Junjie Li, Baoli Li, and Sulong Xu. 2024. A Unified Search and Recommendation Framework Based on Multi-Scenario Learning for Ranking in E-Commerce. In *Proceedings of the 47th International ACM SIGIR Conference*.
- [14] Qi Liu, Atul Singh, Jingbo Liu, Cun Mu, Zheng Yan, and Jan Pedersen. 2024. Long or Short or Both? An Exploration on Lookback Time Windows of Behavioral Features in Product Search Ranking. *arXiv preprint arXiv:2409.17456* (2024). doi:10.48550/arXiv.2409.17456
- [15] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. TensorFlow-Serving: Flexible, High-Performance ML Serving. *arXiv:1712.06139* [cs.DC] <https://arxiv.org/abs/1712.06139>
- [16] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. Beyond Retrieval-Ranking: A Multi-Agent Cognitive Decision Framework for E-Commerce Search. In *Proceedings of the 45th International ACM SIGIR Conference*.
- [17] Zhen Qin, Le Yan, Honglei Zhuang, Yi Tay, Rama Kumar Pasumarthi, Xuanhui Wang, Mike Bendersky, and Marc Najork. 2021. Are Neural Rankers still Outperformed by Gradient Boosted Decision Trees?. In *International Conference on Learning Representations (ICLR)*.
- [18] Redis. 2026. Redis Documentation. <https://redis.io/docs/>
- [19] Scott Rome, Sardar Hamidian, Richard Walsh, Kevin Foley, and Ferhan Ture. 2022. Learning to Rank Instant Search Results with Multiple Indices: A Case Study in Search Aggregation for Entertainment. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Madrid, Spain) (SIGIR '22). Association for Computing Machinery, New York, NY, USA, 3412–3416. doi:10.1145/3477495.3536334
- [20] Tao Yang, Cuize Han, Chen Luo, Parth Gupta, Jeff M. Phillips, and Qingyao Ai. 2024. Mitigating Exploitation Bias in Learning to Rank with an Uncertainty-aware Empirical Bayes Approach. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) (WWW '24). Association for Computing Machinery, New York, NY, USA, 1486–1496. doi:10.1145/3589334.3645487
- [21] Tao Yang, Chen Luo, Hanqing Lu, Parth Gupta, Bing Yin, and Qingyao Ai. 2022. Can Clicks Be Both Labels and Features? Unbiased Behavior Feature Collection and Uncertainty-aware Learning to Rank. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Madrid, Spain) (SIGIR '22). Association for Computing Machinery, New York, NY, USA, 6–17. doi:10.1145/3477495.3531948
- [22] Yisong Yue, Yue Gao, Olivier Chapelle, Ya Zhang, and Thorsten Joachims. 2010. Learning more powerful test statistics for click-based retrieval evaluation. In *Proceedings of the 33rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '10)*. ACM, 507–514. doi:10.1145/1835449.1835534
- [23] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: a unified engine for big data processing. *Commun. ACM* 59, 11 (Oct. 2016), 56–65. doi:10.1145/2934664

A Qualitative Examples

The figures below illustrate qualitative changes after applying the unified learning-to-rank framework to the retailer's product search results. The grid on the left shows the weighted-interleaving baseline, while the grid on the right shows the ranking produced by UR + EF + CL.

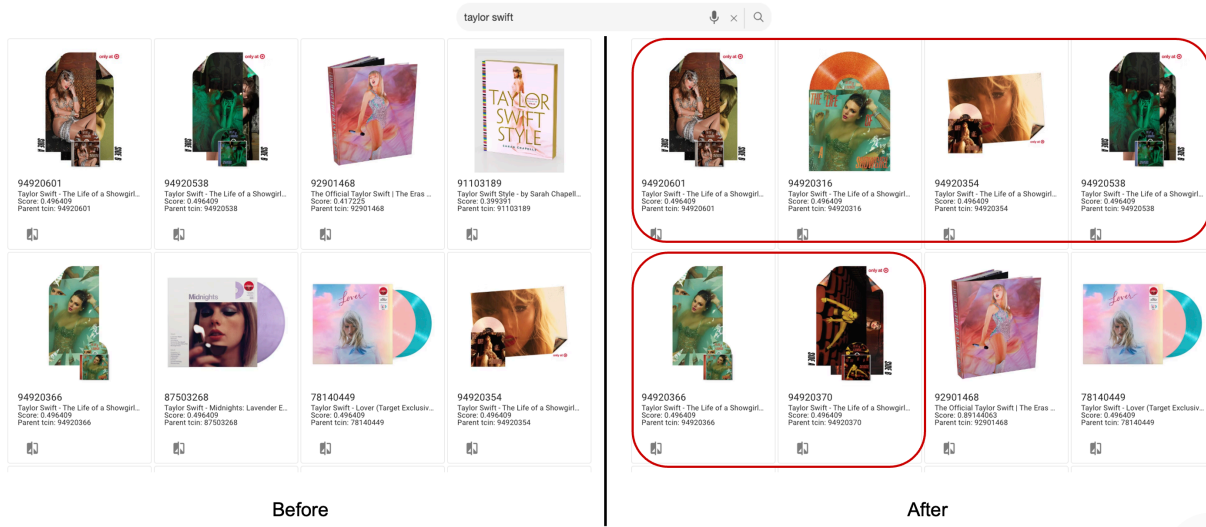


Figure 2: Top 8 ranked results for the query *taylor swift*. The results on the right surface the latest releases by Taylor Swift at the top of the grid because newer items show higher engagement in this category.

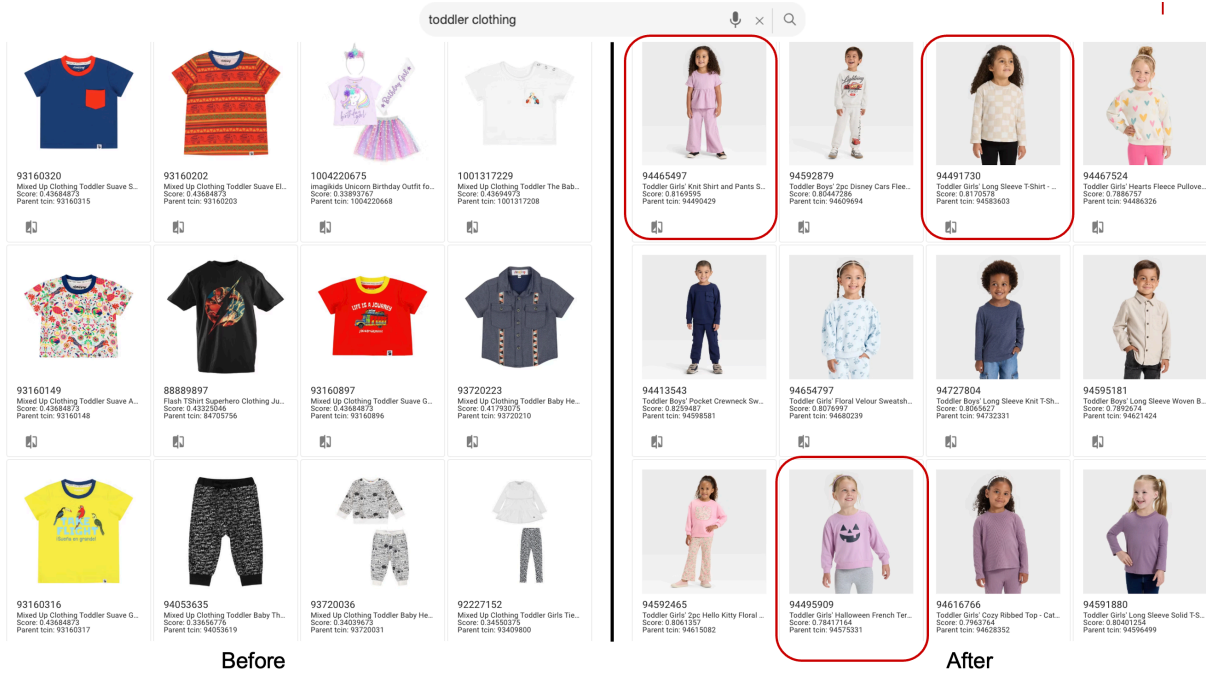


Figure 3: Top 12 ranked results for the query *toddler clothing*. The results after the model update show improved engagement-aware ordering and broader semantic retrieval coverage.